Source Code Listing
CodeAlpha Hangman Game

CodeAlpha Python Programming Internship — Task 1 (2 of 2 tasks completed)

SUMMARY

- Full source of hangman.py, test_hangman.py, and Hangman_Demo.html.
- Companion document: CodeAlpha_PythonProgramming_ResearchPaper.pdf.

Author: Mayank Swaraj (Mayur Krishna)

Repository naming: CodeAlpha_HangmanGame
Date: July 2026

Source File: hangman.py

```
#!/usr/bin/env python3
"""
CodeAlpha Python Programming Task 1 — Hangman Game
A text-based Hangman game: the player guesses a word one letter at a time,
with a limit of 6 incorrect guesses, drawn from a small predefined word list.
"""

import random

WORDS = ["python", "hangman", "keyboard", "internship", "variable"]

HANGMAN_STAGES = [
    """
       ------
       |    |
            |
            |
            |
            |
    ---------""",
    """
       ------
       |    |
       O    |
            |
            |
            |
    ---------""",
    """
       ------
       |    |
       O    |
       |    |
            |
            |
    ---------""",
    """
       ------
       |    |
       O    |
      /|    |
            |
            |
    ---------""",
    """
       ------
       |    |
       O    |
      /|\\   |
            |
            |
    ---------""",
    """
       ------
       |    |
       O    |
      /|\\   |
      /     |
            |
    ---------""",
    """
       ------
       |    |
       O    |
      /|\\   |
      / \\   |
            |
    ---------""",
]

MAX_WRONG_GUESSES = 6


def choose_word(word_list):
```

```
    return random.choice(word_list).lower()


def display_progress(word, guessed_letters):
    return " ".join(letter if letter in guessed_letters else "_" for letter in word)


def play_game(word_list=WORDS, input_func=input, print_func=print, rng_choice=None):
    """Core game loop, with input_func/print_func/rng_choice injectable for automated testing
    without needing real stdin/stdout or true randomness."""
    word = rng_choice(word_list).lower() if rng_choice else choose_word(word_list)
    guessed_letters = set()
    wrong_guesses = 0

    print_func(f"Word to guess has {len(word)} letters. You have {MAX_WRONG_GUESSES} incorrect
      guesses allowed.\n")

    while wrong_guesses < MAX_WRONG_GUESSES:
        print_func(HANGMAN_STAGES[wrong_guesses])
        print_func("Word: " + display_progress(word, guessed_letters))
        print_func(f"Wrong guesses: {wrong_guesses}/{MAX_WRONG_GUESSES}")
        if guessed_letters:
            print_func("Guessed so far: " + ", ".join(sorted(guessed_letters)))

        guess = input_func("Guess a letter: ").strip().lower()

        if len(guess) != 1 or not guess.isalpha():
            print_func("Please enter a single letter.\n")
            continue
        if guess in guessed_letters:
            print_func("You already guessed that letter.\n")
            continue

        guessed_letters.add(guess)

        if guess in word:
            print_func(f"'{guess}' is in the word!\n")
            if all(letter in guessed_letters for letter in word):
                print_func(HANGMAN_STAGES[wrong_guesses])
                print_func("Word: " + display_progress(word, guessed_letters))
                print_func(f"\nYou win! The word was '{word}'.")
                return {"result": "win", "word": word, "wrong_guesses": wrong_guesses}
        else:
            wrong_guesses += 1
            print_func(f"'{guess}' is not in the word. ({wrong_guesses}/{MAX_WRONG_GUESSES}
              wrong)\n")

    print_func(HANGMAN_STAGES[MAX_WRONG_GUESSES - 1])
    print_func(f"\nYou lose! The word was '{word}'.")
    return {"result": "loss", "word": word, "wrong_guesses": wrong_guesses}


def main():
    print("===== HANGMAN =====\n")
    play_game()
    print("\nThanks for playing!")


if __name__ == "__main__":
    main()
```

Source File: test_hangman.py

```
#!/usr/bin/env python3
"""Automated tests for hangman.py — drives the real play_game() function with
scripted input, rather than only testing helper functions in isolation."""

from hangman import play_game, display_progress, WORDS, MAX_WRONG_GUESSES


def make_input_func(scripted_inputs):
    it = iter(scripted_inputs)
    def input_func(prompt=""):
        return next(it)
```

```
    return input_func


def silent_print(*args, **kwargs):
    pass


def test_win_scenario():
    """Guessing every correct letter of a known word should win with zero wrong guesses."""
    word = "python"
    letters_in_order = list(dict.fromkeys(word))  # unique letters, in order
    input_func = make_input_func(letters_in_order)
    result = play_game(word_list=[word], input_func=input_func, print_func=silent_print,
                       rng_choice=lambda wl: wl[0])
    assert result["result"] == "win", f"Expected win, got {result}"
    assert result["wrong_guesses"] == 0, f"Expected 0 wrong guesses, got
      {result['wrong_guesses']}"
    print("test_win_scenario: PASS")


def test_loss_scenario():
    """Guessing 6 letters not in the word should lose with exactly 6 wrong guesses."""
    word = "python"
    wrong_letters = [c for c in "abcdefghijklmnopqrstuvwxyz" if c not in word][:MAX_WRONG_GUESSES]
    assert len(wrong_letters) == MAX_WRONG_GUESSES
    input_func = make_input_func(wrong_letters)
    result = play_game(word_list=[word], input_func=input_func, print_func=silent_print,
                       rng_choice=lambda wl: wl[0])
    assert result["result"] == "loss", f"Expected loss, got {result}"
    assert result["wrong_guesses"] == MAX_WRONG_GUESSES
    print("test_loss_scenario: PASS")


def test_duplicate_guess_does_not_count_twice():
    """Guessing the same wrong letter twice should only count as one wrong guess (the second
    attempt should be rejected as a duplicate, not penalized again)."""
    word = "python"
    wrong_letter = "z"
    # Guess 'z' six times (should count only once as a genuine wrong guess, rest rejected as dup),
    # then guess all real letters to win.
    scripted = [wrong_letter] * 3 + list(dict.fromkeys(word))
    input_func = make_input_func(scripted)
    result = play_game(word_list=[word], input_func=input_func, print_func=silent_print,
                       rng_choice=lambda wl: wl[0])
    assert result["result"] == "win", f"Expected win despite duplicate guesses, got {result}"
    assert result["wrong_guesses"] == 1, f"Expected exactly 1 wrong guess (dupes shouldn't
      recount), got {result['wrong_guesses']}"
    print("test_duplicate_guess_does_not_count_twice: PASS")


def test_invalid_input_does_not_count_as_wrong():
    """Multi-character or non-alphabetic input should be rejected without counting as a wrong
      guess."""
    word = "cat"
    scripted = ["", "12", "ab", "c", "a", "t"]  # 3 invalid attempts, then the 3 real letters
    input_func = make_input_func(scripted)
    result = play_game(word_list=[word], input_func=input_func, print_func=silent_print,
                       rng_choice=lambda wl: wl[0])
    assert result["result"] == "win", f"Expected win, got {result}"
    assert result["wrong_guesses"] == 0, f"Invalid input should not count as a wrong guess, got
      {result['wrong_guesses']}"
    print("test_invalid_input_does_not_count_as_wrong: PASS")


def test_display_progress():
    assert display_progress("cat", set()) == "_ _ _"
    assert display_progress("cat", {"c"}) == "c _ _"
    assert display_progress("cat", {"c", "a", "t"}) == "c a t"
    print("test_display_progress: PASS")


def test_word_list_composition():
    assert len(WORDS) == 5, f"Brief requires exactly 5 predefined words, found {len(WORDS)}"
    assert all(w.isalpha() for w in WORDS)
    print("test_word_list_composition: PASS")


if __name__ == "__main__":
```

```
    test_win_scenario()
    test_loss_scenario()
    test_duplicate_guess_does_not_count_twice()
    test_invalid_input_does_not_count_as_wrong()
    test_display_progress()
    test_word_list_composition()
    print("\nALL TESTS PASSED")
```

Source File: Hangman_Demo.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0">
<title>Hangman — CodeAlpha Python Task 1</title>
<style>
  :root{
    --bg:#12141A; --panel:#1B1F28; --border:#2A303C; --accent:#E8A33D; --good:#4ED08A;
      --bad:#FF6B6B;
    --text:#EDEDF2; --muted:#868FA0;
  }
  *{box-sizing:border-box;}
  html,body{margin:0;padding:0;background:var(--bg);}
  body{font-family:'Inter',system-ui,-apple-system,sans-serif;color:var(--text);display:flex;justi
    fy-content:center;}
  #app{width:100%;max-width:480px;min-height:100vh;padding-bottom:30px;}
  header{padding:20px 18px 12px;}
  .brand{font-family:'Space Grotesk','Inter',sans-serif;font-weight:700;font-size:19px;}
  .brand span{color:var(--accent);}
  .sub{font-size:11px;color:var(--muted);margin-top:4px;}

  #gallows{
    margin:0 18px 14px;background:var(--panel);border:1px solid var(--border);border-radius:14px;
    padding:16px;display:flex;justify-content:center;
  }
  #gallows svg{width:140px;height:160px;}

  #wordDisplay{
    text-align:center;font-family:'JetBrains Mono',monospace;font-size:26px;letter-spacing:6px;
    padding:14px 18px;color:var(--accent);font-weight:700;
  }

  #status{text-align:center;font-size:12.5px;color:var(--muted);padding:0 18px
    14px;min-height:16px;}

  #keyboard{
    display:grid;grid-template-columns:repeat(7,1fr);gap:5px;padding:0 18px 16px;
  }
  .key{
    padding:9px 0;border-radius:7px;border:1px solid var(--border);background:var(--panel);
    color:var(--text);font-weight:700;font-size:12px;cursor:pointer;text-align:center;
  }
  .key:disabled{opacity:0.3;cursor:default;}
  .key.correct{background:rgba(78,208,138,0.18);border-color:var(--good);color:var(--good);}
  .key.wrong{background:rgba(255,107,107,0.18);border-color:var(--bad);color:var(--bad);}

  #controls{padding:0 18px 20px;}
  button.action{width:100%;padding:12px;border-radius:10px;border:none;background:var(--accent);
    color:#221703;font-weight:700;font-size:14px;cursor:pointer;}

  #note{padding:0 18px 30px;font-size:10.5px;color:var(--muted);line-height:1.5;}
</style>
</head>
<body>
<div id="app">
  <header>
    <div class="brand">Hang<span>man</span></div>
    <div class="sub">CodeAlpha Python Task 1 — Hangman Game</div>
  </header>

  <div id="gallows"><svg id="gallowsSvg" viewBox="0 0 140 160"></svg></div>
  <div id="wordDisplay">_ _ _ _ _ _</div>
  <div id="status">Wrong guesses: 0/6</div>
```

```
  |
  |   <div id="keyboard"></div>
  |
  |   <div id="controls">
  |     <button class="action" id="newGameBtn">New Word</button>
  |   </div>
  |
  |   <div id="note">This mirrors hangman.py's exact game logic (5-word list, 6 wrong-guess limit,
  |     duplicate guesses don't count twice) in JavaScript for a clickable browser demo. The real
  |       Python
  |     program was verified with 6 automated tests (win scenario, loss scenario, duplicate-guess
  |     handling, invalid-input handling, display formatting, word-list composition) — see the
  |       research
  |     paper and source code PDF.</div>
  | </div>
  |
  | <script>
  | /* ========================================================
  |    CodeAlpha Python Task 1 — Hangman (JS mirror of hangman.py)
  |    ======================================================== */
  |
  | const WORDS = ["python", "hangman", "keyboard", "internship", "variable"];
  | const MAX_WRONG = 6;
  |
  | let word = "";
  | let guessed = new Set();
  | let wrongCount = 0;
  | let gameOver = false;
  |
  | const GALLOWS_PARTS = [
  |   // cumulative SVG parts revealed at each wrong-guess count (0..6)
  |   '<line x1="10" y1="150" x2="100" y2="150" stroke="#868FA0" stroke-width="3"/><line x1="30"
  |     y1="150" x2="30" y2="10" stroke="#868FA0" stroke-width="3"/><line x1="30" y1="10" x2="90"
  |     y2="10" stroke="#868FA0" stroke-width="3"/><line x1="90" y1="10" x2="90" y2="30"
  |     stroke="#868FA0" stroke-width="3"/>',
  |   '<circle cx="90" cy="45" r="15" fill="none" stroke="#E8A33D" stroke-width="3"/>',
  |   '<line x1="90" y1="60" x2="90" y2="100" stroke="#E8A33D" stroke-width="3"/>',
  |   '<line x1="90" y1="70" x2="70" y2="90" stroke="#E8A33D" stroke-width="3"/>',
  |   '<line x1="90" y1="70" x2="110" y2="90" stroke="#E8A33D" stroke-width="3"/>',
  |   '<line x1="90" y1="100" x2="72" y2="130" stroke="#E8A33D" stroke-width="3"/>',
  |   '<line x1="90" y1="100" x2="108" y2="130" stroke="#E8A33D" stroke-width="3"/>',
  | ];
  |
  | function drawGallows(){
  |   const svg = document.getElementById('gallowsSvg');
  |   let html = '';
  |   for(let i=0;i<=wrongCount;i++) html += GALLOWS_PARTS[i];
  |   svg.innerHTML = html;
  | }
  |
  | function renderWord(){
  |   document.getElementById('wordDisplay').textContent =
  |     word.split('').map(l => guessed.has(l) ? l : '_').join(' ');
  | }
  |
  | function renderKeyboard(){
  |   const kb = document.getElementById('keyboard');
  |   kb.innerHTML = '';
  |   for(let c=97; c<=122; c++){
  |     const letter = String.fromCharCode(c);
  |     const btn = document.createElement('button');
  |     btn.className = 'key';
  |     btn.textContent = letter;
  |     if(guessed.has(letter)){
  |       btn.disabled = true;
  |       btn.className += word.includes(letter) ? ' correct' : ' wrong';
  |     }
  |     if(gameOver) btn.disabled = true;
  |     btn.addEventListener('click', () => guessLetter(letter));
  |     kb.appendChild(btn);
  |   }
  | }
  |
  | function guessLetter(letter){
  |   if(gameOver || guessed.has(letter)) return;
  |   guessed.add(letter);
  |
  |   if(word.includes(letter)){
```

```
    if(word.split('').every(l => guessed.has(l))){
      endGame(true);
    }
  } else {
    wrongCount++;
    if(wrongCount >= MAX_WRONG){
      endGame(false);
    }
  }
  renderWord();
  renderKeyboard();
  drawGallows();
  updateStatus();
}

function updateStatus(){
  const statusEl = document.getElementById('status');
  if(gameOver) return;
  statusEl.textContent = `Wrong guesses: ${wrongCount}/${MAX_WRONG}`;
}

function endGame(won){
  gameOver = true;
  const statusEl = document.getElementById('status');
  statusEl.textContent = won
    ? `You win! The word was "${word}".`
    : `You lose! The word was "${word}".`;
  statusEl.style.color = won ? 'var(--good)' : 'var(--bad)';
  statusEl.style.fontWeight = '700';
  renderKeyboard();
}

function newGame(){
  word = WORDS[Math.floor(Math.random()*WORDS.length)];
  guessed = new Set();
  wrongCount = 0;
  gameOver = false;
  document.getElementById('status').style.color = 'var(--muted)';
  document.getElementById('status').style.fontWeight = 'normal';
  renderWord();
  renderKeyboard();
  drawGallows();
  updateStatus();
}

document.getElementById('newGameBtn').addEventListener('click', newGame);
newGame();
</script>
</body>
</html>
```